OpenArm-v1 및 Foxtech ExoArm-7 연동 구축 보고서

최종 운영 매뉴얼 · 구성 명령어 · 문제 해결 기록

Ubuntu 22.04 / ROS2 Humble / Classic CAN 1 Mbps 기준

목적	Foxtech ExoArm-7 의 관절 입력을 OpenArm-v1 양팔 제어 명령으로 변환
최종 성공 경로	ExoArm ROS2 teleoperator → Foxtech HMI → Python bridge → OpenArm Classic CAN
핵심 결론	OpenArm-v1 은 CAN-FD 가 아니라 Classic CAN 1 Mbps 와 OpenArm(..., False) 조합에서 정상 동작
최종 제어 방식	ExoArm absolute angle 직접 매핑이 아니라 시작 자세 기준 delta mapping 사용
최종 튜닝값	scale = 0.8, gripper trigger 방향 반전(-trigger × 0.5)

작성일: 2026. 07.

목차

- 1. 한눈에 보는 결론
- 2. 시스템을 처음 읽는 사람을 위한 배경 설명
- 3. 개발 환경과 장치 구성
- 4. 최종 운영 절차
- 5. 설치 및 구성 명령어
- 6. OpenArm CAN 문제 복기
- 7. Classic CAN 검증과 zero calibration
- 8. ExoArm ROS2/HMI 확인
- 9. ExoArm → OpenArm bridge 구현
- 10. 튜닝 이력과 최종 파라미터
- 11. Troubleshooting
- 12. 부록: 최종 bridge 코드

읽는 방법: 이 문서는 실행 가능한 최종 절차를 먼저 제시하고, 실패했던 경로는 원인 확인과 재발 방지 목적으로만 정리했다. 그대로 재현할 때는 “최종 운영 절차”와 “설치 및 구성 명령어”를 우선 따른다.

1. 한눈에 보는 결론

이번 구축의 목표는 Foxtech ExoArm-7 을 리더 장치로 사용하고, OpenArm-v1 을 팔로워 로봇으로 제어하는 것이다. 두 장치를 바로 연결하는 방식이 아니라, ExoArm 에서 나오는 ROS2 JointState 값을 Python bridge 가 받아 OpenArm CAN 제어 명령으로 변환하는 구조로 구현했다.

최종 결론: 최종적으로 성공한 조합은 Classic CAN 1 Mbps, oa.OpenArm("can0", False), oa.OpenArm("can1", False), delta mapping, scale 0.8, gripper 방향 반전이다. CAN-FD true 경로와 absolute angle 직접 매핑은 재사용하지 않는다.

구분	최종 상태
OpenArm 통신	Classic CAN 1 Mbps
CAN-FD	최종 제어 경로에서 사용하지 않음
OpenArm Python API	oa.OpenArm("can0", False), oa.OpenArm("can1", False)
ExoArm 입력 토픽	/exo/joint_command [sensor_msgs/msg/JointState]
관절 매핑	시작 자세를 home 으로 저장한 뒤 변화량(delta)만 전달
최종 scale	0.8
Gripper	left/right trigger 값을 음수 방향으로 반전
성공 bridge 파일	~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py

최종 실행 조합

```
# Terminal 1: ExoArm ROS2 websocket teleoperator
cd ~/foxtech_exoarm/qnbot-exo-std-demo
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch qnbot_teleoperator websocket_teleoperator.launch.py

# Terminal 2: Foxtech HMI
cd ~/foxtech_exoarm/software-tool-v1.2.3/linux
./"Qnbot HMI Control-1.2.3.AppImage"

# Terminal 3: ExoArm -> OpenArm bridge
source /opt/ros/humble/setup.bash
python3 ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
```

비상 정지

```
openarm-can-cli -i can0 disable
openarm-can-cli -i can1 disable
```

2. 시스템을 처음 읽는 사람을 위한 배경 설명

2.1 장치의 역할

Foxtech ExoArm-7 은 사람이 움직이는 팔 관절 값을 읽는 리더 장치다. OpenArm-v1 은 실제로 움직이는 팔로워 로봇이다. 리더 장치의 관절각을 그대로 로봇에 넣으면 두 장치의 zero pose, 링크 구조, 관절 부호, 오프셋이 다르기 때문에 로봇이 예상과 다르게 움직일 수 있다.

따라서 중간에 bridge 프로그램이 필요하다. bridge 는 ExoArm 관절값을 받아 OpenArm 에 맞는 목표 관절각으로 바꾸고, 이 값을 CAN 통신으로 OpenArm 모터에 전달한다.

2.2 전체 데이터 흐름

```
Foxtech ExoArm-7
    -> HMI SerialAdapter (/dev/ttyACM0)
```

```
    -> WebSocket forwarding (ws://127.0.0.1:19091)
    -> qnbot_teleoperator ROS2 node
    -> /exo/joint_command (sensor_msgs/msg/JointState, 16 positions)
    -> exo_to_openarm.py bridge
    -> OpenArm can0/can1 Classic CAN 1 Mbps
    -> OpenArm-v1 motors
```

2.3 핵심 용어

용어	의미
Classic CAN	OpenArm-v1 최종 제어에서 사용한 1 Mbps CAN 통신 방식
CAN-FD	공식 예제와 일부 CLI 기본값에서 사용된 방식. 이번 OpenArm-v1 환경에서는 최종 제어 경로로 부적합했음
JointState	ROS2 에서 관절 이름, 위치, 속도, 힘 정보를 담는 메시지 형식
Absolute mapping	ExoArm 의 현재 각도를 OpenArm 목표각으로 그대로 넣는 방식. 이번 구축에서는 이상 동작으로 폐기
Delta mapping	bridge 시작 순간을 기준 자세로 저장하고, 이후 변화량만 OpenArm 에 전달하는 방식
Zero calibration	OpenArm 의 기계적 기준 위치를 모터 쪽에 저장하는 작업

3. 개발 환경과 장치 구성

운영체제	Ubuntu 22.04
ROS	ROS2 Humble
팔로워 로봇	OpenArm-v1
리더 장치	Foxtech ExoArm-7
OpenArm CAN 인터페이스	can0, can1
OpenArm 최종 CAN 설정	Classic CAN 1 Mbps, restart-ms 100
ExoArm 장치 포트	/dev/ttyACM0
HMI 실행 파일	Qnbot HMI Control-1.2.3.AppImage
ExoArm ROS 노드	/websocket_teleoperator
ExoArm 핵심 토픽	/exo/joint_command [sensor_msgs/msg/JointState]
Bridge 위치	~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
백업 권장 파일	~/openarm_teleop/exo_openarm_bridge/exo_to_openarm_WORKING.py

4. 최종 운영 절차

4.1 실행 전 점검

점검 항목	정상 기준
CAN 인터페이스	ip link 에서 can0, can1 확인
OpenArm 전원	모터 전원 ON, E-stop 해제, CAN-H/L 체결
ExoArm HMI	qnbot-exoskeleton Connected, Data Active
ROS2 환경	source /opt/ros/humble/setup.bash 가능
ExoArm 패키지	qnbot_teleoperator, interface_protocol 인식
초기 자세	bridge 시작 순간 ExoArm 과 OpenArm 모두 안전한 중립 자세

4.2 OpenArm CAN 초기화

최종 제어 전에 CAN-FD 기본 configure 를 반복하지 말고, can0 과 can1 을 Classic CAN 1 Mbps 로 명시 설정한다.

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up

sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up

ip -details link show can0
ip -details link show can1
```

4.3 ExoArm teleoperator 실행

```
cd ~/foxtech_exoarm/qnbot-exo-std-demo
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch qnbot_teleoperator websocket_teleoperator.launch.py
```

4.4 Foxtech HMI 실행

```
cd ~/foxtech_exoarm/software-tool-v1.2.3/linux
chmod +x "Qnbot HMI Control-1.2.3.AppImage"
./"Qnbot HMI Control-1.2.3.AppImage"
```

HMI 에서 SerialAdapter 가 /dev/ttyACM0 로 잡히고, qnbot-exoskeleton 이 Connected 상태가 되어야 한다. WebSocket forwarding 은 ws://127.0.0.1:19091 경로로 사용되었다.

4.5 Bridge 실행

```
source /opt/ros/humble/setup.bash
python3 ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
```

처음 콜백에서 Home pose captured 로그가 출력되면, 그 순간의 ExoArm 자세가 기준점으로 저장된다. 이후 OpenArm 목표 각은 기준 자세 대비 변화량에 scale 을 곱해 계산된다.

4.6 성공 파일 백업

```
cp ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py    ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm_WORKING.py
```

5. 설치 및 구성 명령어

5.1 OpenArm CAN 패키지 설치

```
sudo apt update
sudo apt install -y software-properties-common can-utils python3-pip
sudo add-apt-repository -y ppa:openarm/main
sudo apt update
sudo apt install -y openarm-can-utils libopenarm-can-dev

# 설치된 OpenArm CAN 유틸리티 확인
```

```
dpkg -L openarm-can-utils | grep -E "bin|socket|can|configure"
```

5.2 초기 CAN-FD 설정 시도

초기에는 공식 예제 흐름에 맞춰 CAN-FD 설정을 사용했다. 다만 최종적으로 OpenArm-v1 모터가 1 Mbps 로 확인되었기 때문에 이 경로는 운영 절차에 사용하지 않는다.

```
# 초기 시도 경로. 최종 운영에서는 사용하지 않음.
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 dbitrate 5000000 fd on
sudo ip link set can0 up
ip -details link show can0

openarm-can-cli -i can0 can_configure
openarm-can-cli -i can0 discover
openarm-can-cli -i can0 show_param
```

5.3 openarm_can 소스 빌드

```
cd ~
git clone https://github.com/enactic/openarm_can.git
cd ~/openarm_can

# CLI11 누락으로 CMake 오류가 났기 때문에 설치
sudo apt install -y libcli11-dev

mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

빌드 완료 후 openarm-can-cli, openarm-can-demo, openarm-can-motor-sampling-check 실행 파일이 생성된다.

5.4 ExoArm workspace 빌드

```
cd ~/foxtech_exoarm/qnbot-exo-std-demo
source /opt/ros/humble/setup.bash
rosdep update
rosdep install --from-paths . --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
ros2 pkg list | grep -E "qnbot|interface_protocol"
```

5.5 ExoArm 의존성 보완

```
sudo apt install -y python3-pip libfuse2
python3 -m pip install websockets
python3 -m pip install asyncio
python3 -c "import websockets, asyncio; print(websockets.__version__)"
```

websockets 는 teleoperator 실행에 필요했고, libfuse2 는 Ubuntu 22.04 에서 AppImage 실행 오류를 해결하기 위해 설치했다.

6. OpenArm CAN 문제 복기

6.1 초기 실패 증상

처음에는 CAN-FD 경로에서 discover 는 정상적으로 보였지만, 모터 상태 조회와 demo 에서는 실제 위치가 읽히지 않았다. 대표 증상은 ID 가 -1 로 나오거나 position 이 0 으로 고정되는 것이었다.

```
openarm-can-cli -i can0 can_configure
openarm-can-cli -i can0 discover
openarm-can-cli -i can0 show_param
openarm-can-demo
```

대표 실패 출력

```
Arm Motor: 1 ID: -1
Arm Motor: 2 ID: -1
Gripper Motor: 8 ID: -1
position: 0
```

openarm-can-cli monitor 에서도 모든 position, velocity, torque, temperature 가 0 으로 표시되었다. 이 값은 최종 상태 검증에 사용하지 않았다.

6.2 결정적 단서: discover 결과

```
DISCOVERY SUMMARY (Total: 8 motors found)

Send ID     Recv ID     Internal Baudrate Setting
0x01        0x11        1 Mbps (Code: 4)
0x02        0x12        1 Mbps (Code: 4)
0x03        0x13        1 Mbps (Code: 4)
0x04        0x14        1 Mbps (Code: 4)
0x05        0x15        1 Mbps (Code: 4)
0x06        0x16        1 Mbps (Code: 4)
0x07        0x17        1 Mbps (Code: 4)
0x08        0x18        1 Mbps (Code: 4)
```

모터 내부 baudrate 가 1 Mbps 로 확인되었으므로, 5 Mbps CAN-FD 를 전제로 한 예제나 True 플래그로 여는 API 는 현재 OpenArm-v1 환경과 맞지 않았다.

6.3 저수준 CAN 통신과 API 경로의 차이

```
openarm-can-cli -i can0 disable
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000
sudo ip link set can0 up
candump can0

# 다른 터미널
cansend can0 001#FFFFFFFFFFFFFFFC
sleep 0.2
cansend can0 001#FFFFFFFFFFFFFFFD
```

직접 cansend/candump 에서는 001 로 보낸 명령에 대해 011 응답이 확인되었다. 즉 배선과 CAN 인터페이스가 완전히 죽은 상태는 아니었다. 문제는 CAN-FD 로 열리는 OpenArm API/CLI 경로가 현재 모터 상태를 정상적으로 읽지 못한다는 점이었다.

6.4 재발 방지 원칙

구분	내용

반복하지 않을 경로	OpenArm-v1 최종 제어에서 CAN-FD true 예제 그대로 사용
반복하지 않을 경로	시스템 설치본 monitor 의 0 값만 보고 상태 정상 여부 판단
사용할 경로	Classic CAN 1 Mbps 수동 설정
사용할 API	oa.OpenArm("can0", False), oa.OpenArm("can1", False)
검증 도구	~/openarm_can/build/openarm-can-motor-sampling-check

7. Classic CAN 검증과 zero calibration

7.1 demo.cpp 를 Classic CAN 으로 전환

```
cd ~/openarm_can
sed -i 's/OpenArm openarm("can0", true)/OpenArm openarm("can0", false)/'   examples/demo.cpp
cd build
make -j$(nproc)
./openarm-can-demo
```

정상 확인 예

```
Arm Motor: 1 position: 0.049...
Arm Motor: 2 position: -0.044...
```

7.2 can1 도 같은 방식으로 확인

```
cd ~/openarm_can
sed -i 's/OpenArm openarm("can0", false)/OpenArm openarm("can1", false)/'   examples/demo.cpp
cd build
make -j$(nproc)
./openarm-can-demo
```

7.3 zero calibration 스크립트 복사본 생성

원본 /usr/bin 파일을 직접 수정하지 않고, 홈 디렉터리에 복사본을 만들어 True 를 False 로 바꿨다.

```
cp /usr/bin/openarm-can-zero-position-calibration ~/zero_calib_classic.py
sed -i 's/oa.OpenArm(args.canport, True)/oa.OpenArm(args.canport, False)/'   ~/zero_calib_classic.py
chmod +x ~/zero_calib_classic.py
```

7.4 can0/right_arm calibration

```
sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up
~/zero_calib_classic.py --canport can0 --arm-side right_arm
```

7.5 can1/left_arm calibration

```
sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up
~/zero_calib_classic.py --canport can1 --arm-side left_arm
```

정상 종료 출력

```
wrote zero positon to arm
[INFO] Motors disabled, exiting safely.
```

7.6 sampling check

```
cd ~/openarm_can/build

sudo ip link set can0 down
sudo ip link set can0 type can bitrate 1000000 restart-ms 100
sudo ip link set can0 up
./openarm-can-motor-sampling-check --all 100 can0

sudo ip link set can1 down
sudo ip link set can1 type can bitrate 1000000 restart-ms 100
sudo ip link set can1 up
./openarm-can-motor-sampling-check --all 100 can1
```

정상 확인 예

```
ID(S/R) | Position (rad) | Velocity (rad/s) | Torque (Nm) | Temp(C)
1/17    | -0.0002       | -0.0073          | -0.0024     | 29
...
8/24    | -0.0097       | -0.0073          | -0.0073     | 28
```

8. ExoArm ROS2/HMI 확인

8.1 teleoperator 실행과 package not found 해결

ros2 launch 에서 qnbot_teleoperator package not found 가 나온 경우, workspace 가 아직 빌드되지 않았거나 install/setup.bash 가 source 되지 않은 상태였다.

```
cd ~/foxtech_exoarm/qnbot-exo-std-demo
source /opt/ros/humble/setup.bash

# install/이 있으면 먼저 source
source install/setup.bash
ros2 pkg list | grep qnbot
ros2 launch qnbot_teleoperator websocket_teleoperator.launch.py

# install/이 없으면 빌드 후 source
colcon build --symlink-install
source install/setup.bash
ros2 launch qnbot_teleoperator websocket_teleoperator.launch.py
```

8.2 HMI 연결 상태

항목	확인된 값/상태
SerialAdapter	/dev/ttyACM0
Device	qnbot-exoskeleton
상태	Connected, Data Active
WebSocket forwarding	ws://127.0.0.1:19091
주요 확인	Data forwarding target 이 websocket 으로 연결됨

8.3 ROS2 토픽 확인

```
source /opt/ros/humble/setup.bash
source ~/foxtech_exoarm/qnbot-exo-std-demo/install/setup.bash
ros2 node list
ros2 topic list -t
ros2 topic echo /exo/joint_command --once
ros2 topic hz /exo/joint_command
```

확인된 토픽

```
/websocket_teleoperator
/exo/full_state [interface_protocol/msg/ExoFullState]
/exo/gamepad_keys [sensor_msgs/msg/Joy]
/exo/joint_command [sensor_msgs/msg/JointState]
```

8.4 /exo/joint_command 배열 구조

```
left_arm_joint_1~7   = position[0:7]
right_arm_joint_1~7  = position[7:14]
left_trigger_joint   = position[14]
right_trigger_joint  = position[15]
```

bridge 는 이 배열 구조를 기준으로 왼팔, 오른팔, 양쪽 trigger 값을 분리해 사용한다.

9. ExoArm → OpenArm bridge 구현

9.1 초기 absolute mapping 이 실패한 이유

처음에는 ExoArm 의 position 배열을 그대로 OpenArm 목표각으로 사용했다. 이 방식은 두 장치의 zero pose, 관절 부호, 기구학 차이를 반영하지 못해 OpenArm 이 이상하게 움직였다.

```
# 실패 경로: absolute angle 직접 사용
left_q = msg.position[0:7]
right_q = msg.position[7:14]
```

9.2 delta mapping 방식

최종 방식은 bridge 가 시작될 때 ExoArm 의 현재 자세를 home 으로 저장하고, 이후 들어오는 값에서 home 을 뺀 변화량만 OpenArm 에 전달하는 구조다. 이렇게 하면 시작 순간에 팔이 튀는 문제를 줄일 수 있고, 서로 다른 장치의 절대 영점 차이를 직접 맞추지 않아도 된다.

```
target = open_home + scale * sign * (exo_current - exo_home)

scale = 0.8
open_home_left  = [0.0] * 7
open_home_right = [0.0] * 7
left_sign       = [1, 1, 1, 1, 1, 1, 1]
right_sign      = [1, 1, 1, 1, 1, 1, 1]
```

9.3 gripper 제어 추가

초기 bridge 에는 gripper motor 초기화와 trigger 제어 코드가 없었다. 그래서 /exo/joint_command 의 position[14], position[15]가 들어와도 gripper 가 움직이지 않았다. 최종 코드에서는 gripper motor 를 초기화하고 trigger 값을 MIT position command 로 전달한다.

```
self.right.init_gripper_motor(oa.MotorType.DM4310, 8, 24)
self.left.init_gripper_motor(oa.MotorType.DM4310, 8, 24)

left_grip = -msg.position[14] * 0.5
right_grip = -msg.position[15] * 0.5
```

9.4 bridge 파일 생성 및 실행

```
mkdir -p ~/openarm_teleop/exo_openarm_bridge
nano ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
chmod +x ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py

source /opt/ros/humble/setup.bash
python3 ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
```

10. 튜닝 이력과 최종 파라미터

항목	관찰	최종 판단
scale 0.1	움직임이 너무 작음	최종값 아님
scale 10.0	크게 튈 가능성이 있어 위험	최종값 아님
scale 1.0	중간 테스트	최종값 아님
scale 0.3	delta mapping 최초 적용 시 사용	부족함
scale 0.5	중간 테스트	최종값 아님
scale 0.8	사용자가 최종 선택 후 성공 조합 확인	최종값
absolute mapping	방향/오프셋/자세가 맞지 않음	폐기
delta mapping	시작 자세를 home 으로 캡처해 변화량만 전달	최종 방식
gripper +0.5	zero 가 닫힘이라 trigger 를 누르면 더 닫히는 문제	폐기
gripper -0.5	trigger 방향 반전	최종 방식

최종 파라미터

```
scale = 0.8
left_sign = [1, 1, 1, 1, 1, 1, 1]
right_sign = [1, 1, 1, 1, 1, 1, 1]
kp = [20, 20, 12, 12, 5, 5, 4]
kd = [1.0, 1.0, 0.8, 0.8, 0.3, 0.3, 0.3]
left_grip = -msg.position[14] * 0.5
right_grip = -msg.position[15] * 0.5
```

최종 scale 확인 명령

```
sed -i 's/scale = .*/scale = 0.8/'   ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py

grep "scale =" ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py
```

11. Troubleshooting

문제	증상	원인/판단	해결/최종 경로
xarm_ros2 혼동	xArm ROS2 repo 가 후보로 제시됨	UFACTORY xArm 용으로 OpenArm 용이 아님	enactic/openarm_can, openarm_teleop, openarm_ros2 계열 사용
CAN 모터 미검출	discover 에서 모터가 안 잡힘	전원, CAN-H/L, 포트, 배선 점검 필요	can0/can1 각각 discover 수행, 1~8 번 확인
discover 후 show_param 실패	NO RESPONSE FROM MOTOR	discover 결과 1 Mbps Code 4 인데 FD 경로 사용	Classic CAN 1 Mbps + OpenArm(..., False)
openarm-can-demo 실패	ID -1, position 0	demo.cpp 가 OpenArm("can0", true)로 FD 사용	false 로 변경 후 재빌드
CLI11 빌드 오류	Could not find CLI11Config.cmake	개발 패키지 누락	sudo apt install -y libcli11-dev
zero calibration 정지	Enabled... 이후 멈춤	스크립트가 oa.OpenArm(..., True) 사용	복사본 생성 후 True -> False
monitor 값 0	모든 position/temperature 0	시스템 CLI 가 FD 경로로 열림	빌드된 sampling-check 로 Classic CAN 확인
qnbot package not found	ros2 launch 에서 package not found	install/setup.bash source 누락 또는 미빌드	source install/setup.bash 또는 colcon build
HMI AppImage 실패	libfuse.so.2 오류	Ubuntu 22.04 FUSE 라이브러리 누락	sudo apt install -y libfuse2
WebSocket 연결 거부	ECONNREFUSED	teleoperator 보다 HMI 를 먼저 켬	ROS WebSocket 서버 먼저 실행 후 HMI 실행
absolute mapping 이상 동작	방향성/오프셋/스케일이 맞지 않음	두 장치의 zero pose 와 기구학 차이	delta mapping 으로 변경
trigger 미동작	trigger 를 눌러도 gripper 반응 없음	gripper motor init/제어 코드 없음	init_gripper_motor 와 trigger MIT 제어 추가
gripper 방향 반대	trigger 를 누르면 더 닫힘	OpenArm zero 가 gripper 닫힘 상태	left/right_grip 음수 반전

12. 부록: 최종 bridge 코드

아래 코드는 최종 동작이 확인된 bridge 의 핵심 형태다. 실제 파일 위치는 ~/openarm_teleop/exo_openarm_bridge/exo_to_openarm.py 이다.

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import JointState
import openarm_can as oa


class ExoToOpenArm(Node):
    def __init__(self):
        super().__init__("exo_to_openarm")

        self.right = oa.OpenArm("can0", False)
        self.left = oa.OpenArm("can1", False)

        motor_types = [
            oa.MotorType.DM8009, oa.MotorType.DM8009,
            oa.MotorType.DM4340, oa.MotorType.DM4340,
            oa.MotorType.DM4310, oa.MotorType.DM4310,
            oa.MotorType.DM4310,
        ]

        self.right.init_arm_motors(
```

```
            motor_types,
            [1, 2, 3, 4, 5, 6, 7],
            [17, 18, 19, 20, 21, 22, 23],
        )
        self.left.init_arm_motors(
            motor_types,
            [1, 2, 3, 4, 5, 6, 7],
            [17, 18, 19, 20, 21, 22, 23],
        )

        self.right.init_gripper_motor(oa.MotorType.DM4310, 8, 24)
        self.left.init_gripper_motor(oa.MotorType.DM4310, 8, 24)

        self.right.set_callback_mode_all(oa.CallbackMode.STATE)
        self.left.set_callback_mode_all(oa.CallbackMode.STATE)

        self.right.enable_all()
        self.left.enable_all()

        self.sub = self.create_subscription(
            JointState,
            "/exo/joint_command",
            self.cb,
            10,
        )

    def cb(self, msg):
        if len(msg.position) < 14:
            return

        scale = 0.8

        if not hasattr(self, "exo_home_left"):
            self.exo_home_left = list(msg.position[0:7])
            self.exo_home_right = list(msg.position[7:14])
            self.open_home_left = [0.0] * 7
            self.open_home_right = [0.0] * 7
            self.get_logger().info("Home pose captured")
            return

        left_sign = [1, 1, 1, 1, 1, 1, 1]
        right_sign = [1, 1, 1, 1, 1, 1, 1]

        left_q = [
            self.open_home_left[i]
            + scale * left_sign[i] * (msg.position[i] - self.exo_home_left[i])
            for i in range(7)
        ]
        right_q = [
            self.open_home_right[i]
            + scale * right_sign[i] * (msg.position[i + 7] - self.exo_home_right[i])
            for i in range(7)
        ]

        kp = [20, 20, 12, 12, 5, 5, 4]
        kd = [1.0, 1.0, 0.8, 0.8, 0.3, 0.3, 0.3]

        self.left.get_arm().mit_control_all([
            oa.MITParam(kp[i], kd[i], left_q[i], 0.0, 0.0)
            for i in range(7)
        ])
        self.right.get_arm().mit_control_all([
            oa.MITParam(kp[i], kd[i], right_q[i], 0.0, 0.0)
            for i in range(7)
        ])

        if len(msg.position) >= 16:
            left_grip = -msg.position[14] * 0.5
            right_grip = -msg.position[15] * 0.5

            self.left.get_gripper().mit_control_all([
                oa.MITParam(5, 0.3, left_grip, 0.0, 0.0)
            ])
```

```
            self.right.get_gripper().mit_control_all([
                oa.MITParam(5, 0.3, right_grip, 0.0, 0.0)
            ])

        self.left.recv_all()
        self.right.recv_all()

    def destroy_node(self):
        self.left.disable_all()
        self.right.disable_all()
        super().destroy_node()


def main():
    rclpy.init()
    node = ExoToOpenArm()
    try:
        rclpy.spin(node)
    finally:
        node.destroy_node()
        rclpy.shutdown()


if __name__ == "__main__":
    main()
```

마무리

재현에서 가장 중요한 점은 세 가지다. 첫째, OpenArm-v1 은 Classic CAN 1 Mbps 로 연다. 둘째, ExoArm 의 absolute angle 을 그대로 넣지 않고 delta 방식으로 변환한다. 셋째, gripper 는 zero 가 닫힘 상태였기 때문에 trigger 방향을 반전한다. 이 세 가지를 지키면 같은 시행착오를 반복하지 않고 최종 조합을 재현할 수 있다.